University of Stavanger

EXAM IN: STA510 STATISTICAL MODELING AND SIMULATION

DURATION: 4 HOURS DATE: February 24th, 2025
PERMITTED AIDS: Approved simple calculator
THE EXAM CONSISTS OF 3 PROBLEMS ON 5 PAGES, 5 PAGES OF ENCLOSURES.
COURSE RESPONSIBLE: Jörn Schulz PHONE: 51 83 20 28

Problem 1

Consider a random variable X with probability density function

$$f(x) = c \cdot x(1-x)^3, \qquad 0 \leq x \leq 1.$$

a) Show that $c = 20$. (4P)

b) Find $E(X)$ and $E(3X + 2)$. (4P)

c) Describe how we can draw a random sample from the density $f(x)$ using the acceptance-rejection method with a $Beta(2, 2)$ distribution as the proposal distribution. Find potential constants. (6P)

d) Assume that we have drawn a random sample from the density $f(x)$ of length $n = 5000$. How can we estimate the expectation of the random variable X with the density $f(x)$? Given $Var(X) = \frac{2}{63}$, define a 95% confidence interval for the estimate. Below you find a selection of z-scores. (4P)

```
# Some z-scores of the N(0,1) distribution
qnorm(c(0.9,0.95,0.975,0.99))
```

```
## [1] 1.281552 1.644854 1.959964 2.326348
```

e) Assume we want to estimate the integral

$$I = \int_0^1 x(1-x)^3 dx.$$

Write down an expression for the importance sampling estimator $\hat{I}_{IM}$ for I based on n random draws from a $Beta(2, 2)$ distribution. (4P)

Problem 2

Let us consider a distribution, denoted by $\Psi(\alpha, \beta)$, with probability density function $f(x)$ and parameters $\alpha > 0$ and $\beta > 0$ defined by

$$f(x) = \frac{\alpha}{\beta^\alpha} x^{\alpha-1} \mathbb{I}(0 < x < \beta) \tag{1}$$

where $\mathbb{I}$ is the indicator function with

$$\mathbb{I}(0 < x < \beta) = \begin{cases} 1 & 0 < x < \beta \\ 0 & \text{otherwise.} \end{cases}$$

a) Show that the cumulative density function is given by $F(x) = \frac{x^\alpha}{\beta^\alpha}$ for $0 < x < \beta$. (4P)

b) Given $\alpha = 3$ and $\beta = 5$, find $P(X < 2)$ and $P(2 < X < 4)$. (4P)

c) Find the inverse cumulative distribution function. Explain (with words and formulas, not code) how we can simulate $n = 5000$ random numbers from this distribution using the inverse transformation method. (4P)

Let us assume we have observations $\mathbf{x} = (x_1, \ldots, x_n)$ of sizes of oil reserves and $X_i \overset{\text{iid}}{\sim} Pareto(a, b)$ with density

$$p(x|a, b) = \frac{ba^b}{x^{b+1}} \mathbb{I}(a < x), \quad \text{for } 0 < a \text{ and } 0 < b.$$

We want to estimate the parameters a and b of the Pareto distribution by a Bayesian approach using a simple prior $p(a, b) \propto \mathbb{I}(a, b > 0) = \mathbb{I}(a > 0)\mathbb{I}(b > 0)$ where $\mathbb{I}$ is the indicator function similar as defined above. Note: If you find 2d-f difficult, you can solve 2g-h given the solutions in 2e-f.

d) Show that the posterior can be written as

$$p(a, b|\mathbf{x}) \propto \frac{b^n a^{nb}}{(\prod_{i=1}^n x_i)^{b+1}} \mathbb{I}(a < x^*)\mathbb{I}(a, b > 0)$$

where $x^* = \min\{x_1, \ldots, x_n\}$. (4P)

e) Show that the conditional posterior $a|b, \mathbf{x} \propto \Psi(a|nb + 1, x^*)$ where the distribution $\Psi(\alpha, \beta)$ and its corresponding density function is defined above in (1). (3P)

f) Show that the conditional posterior

$$b|a, \mathbf{x} \propto Gamma\left(b|n + 1, \left(\sum \ln(x_i) - n \ln(a)\right)^{-1}\right). \quad (4P)$$

g) Explain (with words and formulas, no code) the algorithm of a Gibbs sampler. (4P)

h) Assume the state vector θ in the Gibbs sampler is high dimensional (not only two dimenstional as above). Describe briefly three points for implementing an efficient Gibbs sampler. (3P)

Problem 3

Let X be a random variable that is $Pareto(a, b)$ distributed with parameters $a > 0$ and $b > 0$ and probability density function

$$f(x) = \frac{ba^b}{x^{b+1}}, \quad \text{for } 0 < a < x \text{ and } 0 < b.$$

Assume that the parameter $a > 0$ is known, i.e. only the parameter b is unknown.

a) Assume we have a random sample $x_1, \ldots, x_n$ of X and we want to estimate the parameter b based on the observed sample. Show that the Maximum Likelihood Estimator (MLE) for b is

$$\hat{b}_{MLE} = \frac{n}{\sum_{i=1}^{n} \ln(\frac{x_i}{a})}. \quad (6P)$$

b) It can be shown (you don't need to do that) that

$$E(\hat{b}_{MLE}) = \frac{nb}{n-1} \text{ and } E(\hat{b}^2_{MLE}) = \frac{n^2b^2}{(n-1)(n-2)}.$$

Is $\hat{b}_{MLE}$ an unbiased estimator? Justify your answer! (2P)
Find $Var(\hat{b}_{MLE})$! (2P)

c) Let consider an alternative estimator $b^* = \frac{n-1}{\sum_{i=1}^{n} \ln(\frac{x_i}{a})}$.
Find $E(b^*)$ and $Var(b^*)$! (4P)
Which one is the better estimator $\hat{b}_{MLE}$ or b^*? Justify your answer! (2P)

d) Assuming $a = 2$, below you see some code to implement the bootstraping algorithm in order to estimate the distribution of the estimators given 40 observations. Which are correct implementations of the bootstrap algorithm (bootSample1 - bootSample4)? Justify your answer! (8P)

```
data <- c(4.71,  27.81,   5.27,  33.47,   2.83,  20.42,   3.05,  45.26,
          2.11,   3.24,   5.36,   2.14,   3.45,  14.86,   4.01,  11.52,
          4.98,   5.83,   2.62,  81.57,   2.33,   5.17,   3.13,  13.51,
          9.32,  32.08,   2.07, 694.32,   5.51,   4.50,   2.00, 133.31,
          3.51,  14.22,   2.09,   2.07,  26.07,   2.35,  39.44,  13.95)

a <- 2
n <- length(data)

b.MLE <- function(x){
  theta <- n / (sum( log(x/a) ))
  return(theta)
}
```

```r
b.Star <- function(x){
  theta <- (n-1) / (sum( log(x/a) ))
  return(theta)
}


# Bootstrap version 1
bootSample1 <- function(data,nrep,bootFunc){
  bootEstB <- numeric(nrep)
  for(i in 1:nrep){
    bootEstB[i] <- bootFunc(sample(data,size=length(data),replace=FALSE))
  }
  return(bootEstB)
}
bootEstB <- bootSample1(data=data,nrep=5000,bootFunc=b.MLE)

# Bootstrap version 2
bootSample2 <- function(data,nrep,bootFunc){
  bootEstB <- numeric(nrep)
  for(i in 1:length(data)){
    bootEstB[i] <- bootFunc(sample(data,size=nrep,replace=TRUE))
  }
  return(bootEstB)
}
bootEstB <- bootSample2(data=data,nrep=5000,bootFunc=b.Star)

# Bootstrap version 3
bootSample3 <- function(data,nrep,bootFunc){
  bootEstB <- numeric(nrep)
  for(i in 1:nrep){
    bootEstB[i] <- bootFunc(sample(data,size=length(data),replace=TRUE))
  }
  return(bootEstB)
}
bootEstB <- bootSample3(data=data,nrep=5000,bootFunc=b.MLE)

# Bootstrap version 4
bootSample4 <- function(data,nrep,bootFunc){
  bootEstB <- numeric(nrep)
  for(i in 1:nrep){
    bootEstB[i] <- mean(sample(data,size=length(data),replace=TRUE))
  }
  return(bootEstB)
}
bootEstB <- bootSample4(data=data,nrep=5000,bootFunc=b.Star)
```

e) Below you see some code to estimate the mean square error of the MLE. Which are correct implementations? Justify your answer! (3P)

```
# Mean square error
b.mse1 <- var(bootEstB)-(mean(bootEstB) - b.MLE(data))^2
b.mse2 <- sd(bootEstB)-(mean(bootEstB) - b.MLE(data))^2
b.mse3 <- sd(bootEstB)-(mean(bootEstB))^2
```

f) Below you see some code to estimate the 95% normal bootstrap confidence and the 95% percentile bootstrap interval for the MLE. Which are correct implementations? Justify your answer! (4P)

```
# Standard normal bootstrap interval 1
normalBootCI1 <- b.MLE(data)+c(-qnorm(0.95)*sd(bootEstB),
                               qnorm(0.95)*sd(bootEstB))

# Standard normal bootstrap interval 2
normalBootCI2<- b.MLE(data)+c(-qnorm(0.975)*var(bootEstB),
                              qnorm(0.975)*var(bootEstB))

# Percentile bootstrap interval 1
percentBootCI <- quantile(bootEstB,probs=c(0.025,0.975))

# Percentile bootstrap interval 2
percentBootCI <- 1-quantile(bootEstB,probs=c(0.025,0.975))
```